Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: CSA0603 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E/B.Tech-Computer Science and Engineering
Course Code & Course Name	CSA0603-Design and Analysis of Algorithms
Academic Year / Batch	2024
Faculty Name	Dr.P. Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search
Date of Issue	01-09-2026
Date of Submission	02-09-2025
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education: Develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure – Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.
Industry / Societal Relevance	Promotes innovation and development of computational techniques for solving real-world spatial and engineering problems.

B. Assignment Problem / Challenge

Given the set of 12 numbers A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}, apply the Brute Force technique to solve the Sorting and Searching problems. For Selection Sort, repeatedly identify the smallest element from the unsorted portion and place it in its correct position. Show the major intermediate steps, determine the sorted sequence, develop suitable algorithm/pseudocode, and analyze the number of comparisons and swaps performed. For Sequential Search, search for the key value 68 by checking elements one by one from the beginning of the list.

ASSESSMENT: Brute Force Sorting and Searching with Real-World Application

Title

Application and Performance Analysis of Brute Force Sorting and Searching Algorithms in Smart Energy Data Management

A. PROBLEM STATEMENT

1. Introduction

- Modern smart homes and electricity-monitoring systems continuously generate large amounts of numerical data. This data must be organized and searched efficiently to understand energy-consumption patterns.
- Consider a smart energy-monitoring system in which each numerical value represents a household electricity-consumption measurement.
- The initial set of measurements is:

A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}

- The system needs to perform two fundamental operations:

1. Sorting: Arrange the measurements in ascending order using Selection Sort.
2. Searching: Locate the measurement 68 using Sequential Search.

- The problem is then extended to large real-world datasets to compare the performance of Brute Force algorithms with more efficient algorithms.

2. Real-World Scenario

- Imagine a Smart Household Energy Monitoring System.
- Smart meters continuously record electricity consumption. The collected values can be processed by software to:
- organize energy readings,
- locate specific readings,
- analyze consumption,
- identify unusually high consumption,

- generate reports,
- support energy-saving decisions.
- For a small number of records, simple Brute Force algorithms may be sufficient.
- However, smart meters can generate millions of records, making algorithm efficiency extremely important.
- Therefore, this assessment compares:

Sorting

- Selection Sort – Brute Force
- Merge Sort – Divide and Conquer
- Hybrid Sort – practical hybrid approach

Searching

- Sequential Search – Brute Force
- Binary Search – efficient searching after sorting

3. Objectives

- The objectives of this assessment are:
- Apply Brute Force Selection Sort.
- Apply Brute Force Sequential Search.
- Show the major intermediate sorting steps.
- Find the position of key 68.
- Count comparisons and swaps.
- Distinguish comparisons from data movements.
- Derive the number of Selection Sort comparisons.
- Analyze best, average and worst cases.
- Express complexity using Big-O, Big-Ω and Big-Θ.
- Study the effect of increasing input size.

- Compare Brute Force and efficient algorithms.
- Use a real-world numerical dataset.
- Experiment with input sizes from $10^3$to 10^6.
- Plot execution time and operation counts.
- Relate experimental results to theoretical complexity.
- Discuss sustainability and SDG contributions.

4. Input and Output

Input

A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}

- Search key:

68

Expected sorted output

5 8 11 17 21 26 34 42 55 68 79 93

Expected search result

68 is found at the 8th position.

- Using zero-based indexing:

Index = 7

5. Selection Sort – Brute Force Technique

- Selection Sort repeatedly searches the unsorted portion of the array and identifies the smallest element.
- □genui□{"learning_viz":{"type_id":"SELECTION_SORT"}}□
- Initially:

42 17 8 93 26 55 11 68 34 5 79 21

Pass 1

- Smallest element = 5

5 17 8 93 26 55 11 68 34 42 79 21

Pass 2

- Smallest = 8

5 8 17 93 26 55 11 68 34 42 79 21

Pass 3

- Smallest = 11

5 8 11 93 26 55 17 68 34 42 79 21

Pass 4

- Smallest = 17

5 8 11 17 26 55 93 68 34 42 79 21

Pass 5

- Smallest = 21

5 8 11 17 21 55 93 68 34 42 79 26

Pass 6

- Smallest = 26

5 8 11 17 21 26 93 68 34 42 79 55

Pass 7

- Smallest = 34

5 8 11 17 21 26 34 68 93 42 79 55

Pass 8

- Smallest = 42

5 8 11 17 21 26 34 42 93 68 79 55

Pass 9

- Smallest = 55

5 8 11 17 21 26 34 42 55 68 79 93

Pass 10

- Smallest = 68

5 8 11 17 21 26 34 42 55 68 79 93

Final Sorted Sequence

5 8 11 17 21 26 34 42 55 68 79 93

6. Selection Sort Algorithm / Pseudocode

```
SelectionSort(A, n)

FOR i = 0 TO n-2

   minIndex = i

   FOR j = i+1 TO n-1

      IF A[j] < A[minIndex]

         minIndex = j

   IF minIndex != i

      SWAP A[i] and A[minIndex]

END FOR
```

Working principle

Unsorted Array

↓

Find minimum

↓

Place minimum at beginning

↓

Reduce unsorted portion

↓

Repeat

↓

Sorted Array

7. Selection Sort Comparisons

- For $n$elements:

$$(n-1)+(n-2)+(n-3)+\ldots+1$$

- Using the summation formula:

$$C(n)=\frac{n(n-1)}{2}$$

- For $n=12$:

$$C(12)=\frac{12(12-1)}{2}$$
$$=\frac{12\times 11}{2}$$
$$\boxed{66}$$

- Therefore:
- Total comparisons = 66

8. Swaps and Data Movements

- For the given input, Selection Sort performs:

$$\boxed{10 \text{ swaps}}$$

- The important distinction is:

Comparison

- Checking whether:

A[j] < A[minIndex]

Swap

- Moving the selected minimum into its correct position.
- Thus:

Comparisons = 66

Swaps = 10

- A swap can involve several underlying assignments, so number of swaps and number of data movements are not the same thing.

9. Sequential Search

- The search key is:

Key = 68

- Original array:

42 17 8 93 26 55 11 68 34 5 79 21

- The algorithm checks elements from the beginning.

Step	Element	Result
1	42	Not Found
2	17	Not Found
3	8	Not Found
4	93	Not Found
5	26	Not Found
6	55	Not Found
7	11	Not Found
8	68	Found

- Therefore:

$$\boxed{\text{Position} = 8}$$

- and

$$\boxed{\text{Comparisons} = 8}$$

- Using zero-based indexing:

$$\boxed{\text{Index} = 7}$$

10. Sequential Search Pseudocode

```
SequentialSearch(A, n, key)

FOR i = 0 TO n-1

   IF A[i] == key

      RETURN i

RETURN -1
```

11. Sequential Search Complexity

Best Case

- The key is the first element.

68 42 17 ...

- Only one comparison is required.

$$\boxed{\Omega(1)}$$

Average Case

- The key is approximately in the middle.
- Approximately:

$$n/2$$

- comparisons.
- Therefore:

$$\boxed{\Theta(n)}$$

Worst Case

- The key is the final element or is absent.

$$n$$

- comparisons.
- Therefore:

$$\boxed{O(n)}$$

12. Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	O(1)
Sequential Search	$\Omega(1)$	$\Theta(n)$	O(n)	O(1)
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	O(n)
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$	O(1)*

- *For iterative Binary Search.

13. Effect of Increasing Input Size

- Selection Sort has quadratic growth.

$$C(n) = \frac{n(n-1)}{2}$$

Input Size	Selection Sort Comparisons
10	45
100	4,950
1,000	499,500
10,000	49,995,000
100,000	4,999,950,000
1,000,000	499,999,500,000

- At one million records, Selection Sort requires approximately:

500 billion comparisons

- This clearly demonstrates why the Brute Force approach becomes inefficient for large datasets.

14. Sequential Search as n Increases

n	Worst-Case Comparisons
10	10
1,000	1,000
10,000	10,000
100,000	100,000
1,000,000	1,000,000

- Sequential Search grows linearly, so it is considerably better than Selection Sort, but repeated searches over millions of records can still become expensive.

B. REAL-WORLD DATASET AND EXPERIMENT

15. Publicly Available Dataset

- For the large-scale experiment, use the UCI Individual Household Electric Power Consumption Dataset.
- The dataset contains 2,075,259 measurements recorded at one-minute intervals over almost four years. It includes numerical variables such as Global_active_power, Voltage, and Global_intensity.
- This makes it suitable for testing input sizes from approximately:

$10^3 \rightarrow 10^4 \rightarrow 10^5 \rightarrow 10^6$

- records.

Dataset Source

- UCI Individual Household Electric Power Consumption Dataset

16. Dataset Preprocessing

- Use:

Global_active_power

- as the numerical input.

- Preprocessing:

Raw Dataset

↓

Load CSV

↓

Select Global_active_power

↓

Remove missing values

↓

Convert values to numerical format

↓

Create subsets

↓

1,000

10,000

100,000

1,000,000

- The same subset must be supplied to all sorting algorithms to ensure a fair comparison.

17. Algorithms for Experimental Comparison

Sorting

Brute Force

Selection Sort

Divide and Conquer

Merge Sort

Hybrid

IntroSort

- or a

Merge Sort + Insertion Sort hybrid

- with a documented threshold.
- For example:

If partition size <= 16

use Insertion Sort

Else

use Merge/Divide-and-Conquer method

18. Searching

- Compare:

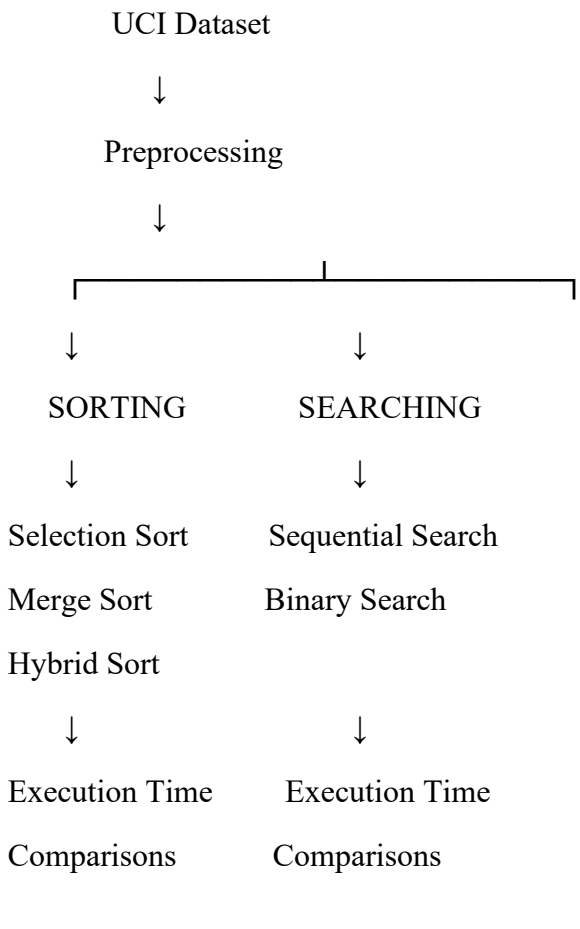
Sequential Search

- with:

Binary Search

- Binary Search must be performed on sorted data.

19. Experimental Methodology



Swaps Search Operations

↓ ↓

Performance Analysis

20. Experimental Input Sizes

- Use:

n = 1,000

n = 10,000

n = 100,000

n = 1,000,000

- For every n, use the same data subset for Selection Sort, Merge Sort and Hybrid Sort.
- Run the experiments multiple times and record the execution time.

21. Tools and Environment

- The implementation can use:
- Python
- VS Code / Jupyter Notebook / Google Colab
- Python time.perf_counter() for execution-time measurement
- Matplotlib for graphs
- Pandas for dataset preprocessing
- Document the actual:
- CPU,
- RAM,
- operating system,
- Python version,
- dataset version/source,
- hybrid threshold.

22. Results Table – Sorting

- Actual execution times should be filled using experimental measurements.

n	Selection Sort	Merge Sort	Hybrid Sort
1,000	Measured	Measured	Measured
10,000	Measured	Measured	Measured
100,000	Measured	Measured	Measured
1,000,000	Extremely high / impractical	Measured	Measured

- Do not invent execution times; use the values obtained from your system.

23. Results Table – Searching

n	Sequential Search	Binary Search
1,000	Measured	Measured
10,000	Measured	Measured
100,000	Measured	Measured
1,000,000	Measured	Measured

24. Required Graphs

Graph 1: Input Size vs Execution Time

- Plot:

X-axis → Input Size

Y-axis → Execution Time

- Compare:
- Selection Sort
- Merge Sort
- Hybrid Sort

- The Selection Sort curve is expected to increase much faster.

Graph 2: Input Size vs Operation Count

- Compare:
- Selection Sort comparisons
- Sequential Search comparisons
- Binary Search comparisons
- Expected trend:

Selection Sort → n^2

Sequential Search → n

Binary Search → log n

C. COMPARISON OF BRUTE FORCE APPROACHES

Feature	Selection Sort	Sequential Search
Problem	Sorting	Searching
Strategy	Repeatedly find minimum	Check elements one by one
Technique	Brute Force	Brute Force
Basic operation	Comparison	Equality comparison
Best case	$\Theta(n^2)$	$\Theta(1)$
Average	$\Theta(n^2)$	$\Theta(n)$
Worst	$\Theta(n^2)$	$\Theta(n)$
Extra space	O(1)	O(1)
Large datasets	Very inefficient	Can become slow
Main advantage	Simple and in-place	Works on unsorted data

25. Limitations

Selection Sort

- Selection Sort becomes inefficient when:
- $n$is very large,
- sorting is performed frequently,
- real-time performance is required,
- millions of records must be processed.
- Its $O(n^2)$growth is the major limitation.

Sequential Search

- Sequential Search becomes inefficient when:
- there are millions of records,
- many searches are performed,
- fast response is required.
- Its $O(n)$worst-case growth becomes expensive for large datasets.

26. Possible Improvements

For Sorting

- Replace:

Selection Sort → O(n²)

- with:

Merge Sort → O(n log n)

- or a practical hybrid such as:

IntroSort / Merge + Insertion hybrid

For Searching

- Replace:

Sequential Search → O(n)

- with:

Binary Search → O(log n)

- Condition: The data must first be sorted.

D. STUDENT WORK

27. Problem Understanding

Input

42 17 8 93 26 55 11 68 34 5 79 21

Sorted Output

5 8 11 17 21 26 34 42 55 68 79 93

Search Key

68

Search Result

Position = 8

Index = 7

Comparisons = 8

28. Application of Course Knowledge

- The assessment applies:

Principles

- Brute Force
- Divide and Conquer
- Algorithm selection

Theories

- Algorithm analysis
- Asymptotic analysis

- Computational complexity

Mathematical Models

$$\frac{n(n-1)}{2}$$

- for Selection Sort comparisons.

Algorithms

- Selection Sort
- Sequential Search
- Merge Sort
- Binary Search
- Hybrid Sort

Engineering Concepts

- Scalability
- Performance
- Resource utilization
- Algorithm selection
- Trade-off analysis

29. Design / Methodology

- The proposed system consists of:

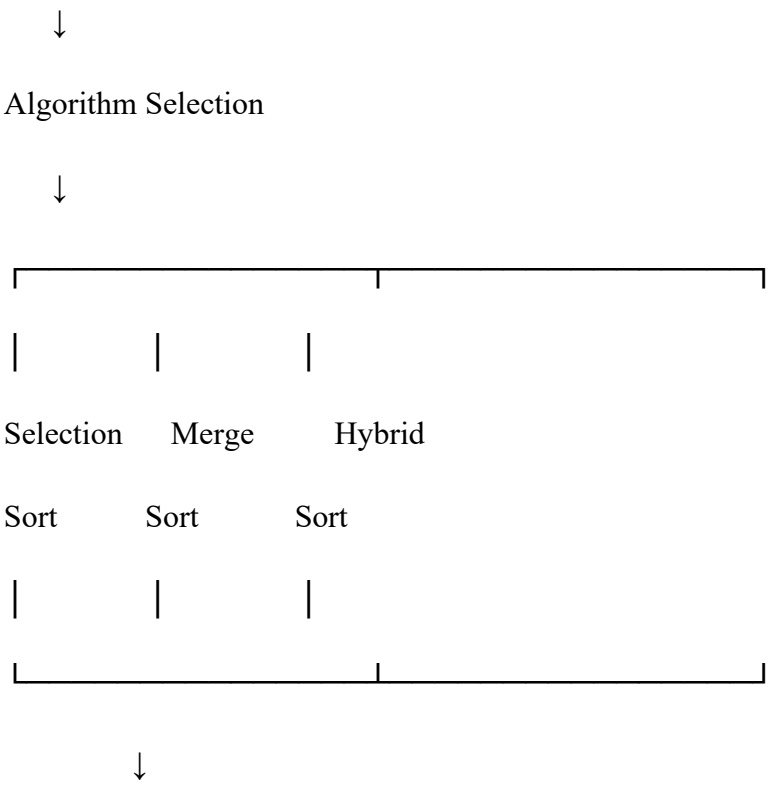
Data Collection

↓

Data Cleaning

↓

Numerical Dataset



30. Validation

- The solution is validated using:
- sorted output,
- search result,
- comparison counts,
- swap counts,
- execution time,
- operation counts,
- graphs,
- theoretical complexity.

- For the original dataset:

Selection Sort:

Comparisons = 66

Swaps = 10

Sequential Search:

Key = 68

Position = 8

Comparisons = 8

- These values can be manually verified.

31. Engineering Decision

- For a small dataset:

n = 12

- Selection Sort and Sequential Search are attractive because they are:
- simple,
- understandable,
- easy to implement,
- require little extra memory.
- For a large dataset:

n = 1,000,000

- Selection Sort is not an appropriate choice because of its quadratic growth.
- A better architecture is:

Large Dataset

↓

Merge/Hybrid Sort

↓

Sorted Dataset

↓

Binary Search

- This provides much better scalability.

32. Broader Considerations

Sustainability

- Efficient algorithms can reduce unnecessary computational work when processing very large datasets.

Environment

- Lower computational requirements can potentially reduce energy consumption in large-scale computing environments.

Society

- Energy-monitoring systems can help households understand their electricity consumption and make informed decisions.

Economics

- Faster algorithms can reduce:
- processing time,
- computing resources,
- cloud infrastructure costs,
- operational delays.

Ethics

- Energy data can reveal information about household behavior. Therefore, a real system should ensure:
- data privacy,
- secure storage,
- responsible data usage,
- appropriate access controls.

Accessibility

- Energy-monitoring applications should present information in a simple and understandable

way so that users with different technical backgrounds can benefit.

33. SDG Contribution

he real-world energy-monitoring application connects the project with the following Sustainable Development Goals.

SDG 4 – Quality Education

- The project develops **analytical thinking, programming skills, and computational problem-solving abilities** through the implementation of sorting and searching algorithms.
- Students learn to analyze **comparisons, swaps, execution time, and algorithm complexity**, connecting theoretical concepts with practical applications.

SDG 9 – Industry, Innovation and Infrastructure

- The project applies **algorithms and computational techniques** to a real-world energy-monitoring and data-processing problem.
- Efficient sorting and searching methods demonstrate how computational solutions can support **smart systems, IoT applications, and scalable digital infrastructure**.

SDG Summary

SDG	Contribution
SDG 4	Develops analytical thinking, programming, and computational problem-solving skills
SDG 9	Applies algorithms and computational techniques to real-world smart infrastructure and data-processing systems

34. Final Conclusion

- This assessment demonstrates how Brute Force algorithms can solve fundamental sorting and searching problems and how their performance changes when the input size increases.
- For the given dataset:

42 17 8 93 26 55 11 68 34 5 79 21

- Selection Sort produces:

5 8 11 17 21 26 34 42 55 68 79 93

- with:

$$\boxed{66 \text{ comparisons}}$$

- and:

$$\boxed{10 \text{ swaps}}$$

- Sequential Search finds 68 at the:

$$\boxed{8^{th} \text{ position}}$$

- after:

$$\boxed{8 \text{ comparisons}}$$

- The theoretical analysis shows that Selection Sort has:

$$\boxed{\Theta(n^2)}$$

- growth, while Sequential Search has:

$$\boxed{\Theta(n)}$$

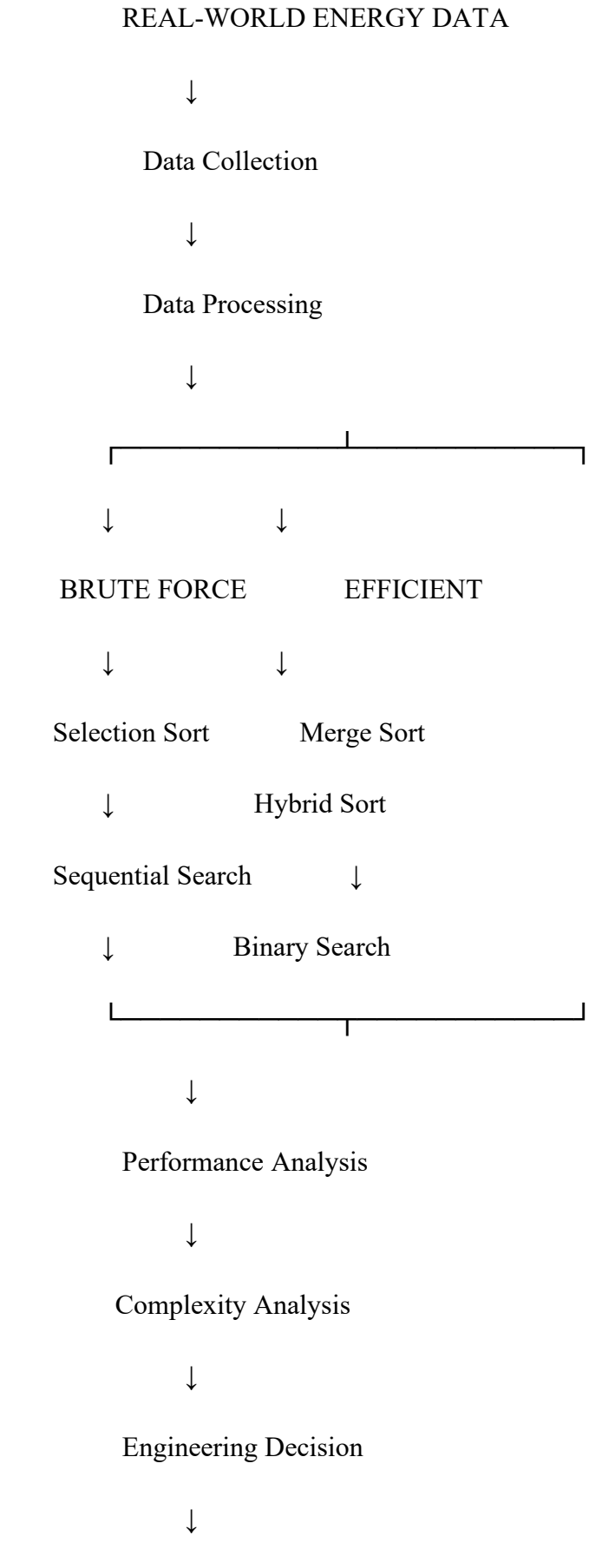
- growth in its average/worst-case behavior.
- As the input increases toward one million records, Selection Sort becomes impractical because its number of comparisons approaches 500 billion. Sequential Search is less expensive but can still become slow for repeated searches over very large datasets.
- Therefore, practical large-scale systems should use:

$$\boxed{\text{Merge/Hybrid Sort + Binary Search}}$$

- where appropriate.
- The real-world smart energy-monitoring application also demonstrates how algorithmic

efficiency can support energy management, responsible resource consumption and sustainable digital infrastructure, connecting the technical work to SDG 7, SDG 9, SDG 12 and SDG 13.

Overall flow of the assessment



Sustainable Application

↓

SDG 7, 9, 12 and 13

- This combined structure covers A. Problem Statement, B. Requirements and Constraints, C. Experimental work, D. Student Work, course knowledge, algorithms, mathematical derivation, tools, validation, graphs, engineering decisions, real-world application, and broader SDG considerations in one continuous assessment.

C.Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.

		appropriately for solution development.			
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.

		scalability with clear evidence.			
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload